CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

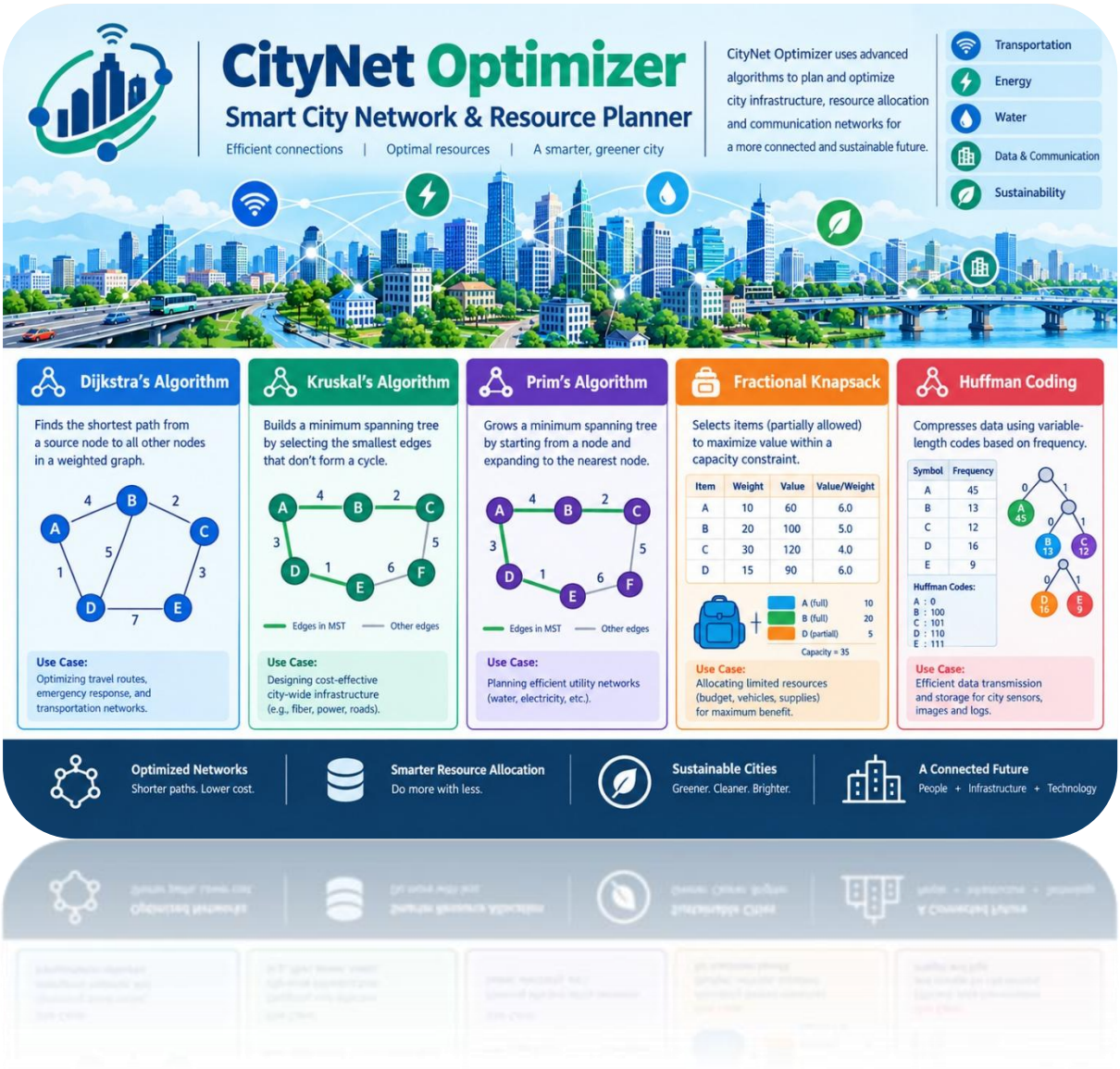
UNIVERSITY INSTITUTE OF COMPUTING
Discover. Learn. Empower.

A MINI PROJECT REPORT ON

CityNet Optimizer

Smart City Network & Resource Planner

using Dijkstra, Kruskal, Prim, Fractional Knapsack and Huffman Coding



Subject: Design and Analysis of Algorithms (DAA)

Subject Code: 26CAP-612

Submitted by

ADITYA MANGALAM

Under the guidance of

Mr. RAJAN SALUJA

Academic Year: 2026 – 2027

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF COMPUTING
Discover. Learn. Empower.

CERTIFICATE

This is to certify that the mini project work entitled **"CityNet Optimizer – Smart City Network & Resource Planner"** has been successfully carried out by ADITYA MANGALAM, UID: 26MCC20078 of Semester-I, Department of Cloud Computing & DevOps, Chandigarh University, in partial fulfilment of the requirements for the subject **Design and Analysis of Algorithms** during the academic year 2026 – 2027.

The project report has been approved as it satisfies the academic requirements in respect of the mini project prescribed for the said subject.

Project Guide
Mr. RAJAN SALUJA

Head of Department
Dr. AMANPREET KAUR SANDHU

Date: ____ / ____ / 2026

Place: Chandigarh University, Gharuan, Mohali

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who helped me in the successful completion of this mini project, “CityNet Optimizer”.

First and foremost, I thank my project guide, Mr. RAJAN SALUJA, for the valuable guidance, constant encouragement and constructive suggestions given throughout the development of this project. Their explanation of the algorithm design paradigms in the Design and Analysis of Algorithms course was the foundation of this work.

I am thankful to Head of the Department of Master of Computer Application, and to the Director of Chandigarh University, for providing the laboratory facilities and a supportive academic environment.

I also thank all the teaching and non-teaching staff of the department, my friends and classmates for their help and feedback, and my parents for their continuous motivation and support.

ADITYA MANGALAM
UID: 26MCC20078

ABSTRACT

Modern cities depend on efficient networks – roads for emergency vehicles, cables for communication, and logistics for relief supplies. Planning these networks manually is slow and expensive, and it is exactly the kind of problem that algorithm design techniques are built to solve.

CityNet Optimizer is a desktop application written in Python (Tkinter) that models ten cities of Punjab and Chandigarh as a weighted graph and solves four real-world planning problems on it. (1) **Dijkstra's algorithm** finds the shortest emergency route between any two cities. (2) **Kruskal's and Prim's algorithms** compute the minimum-cost optical-fibre network that connects all cities, saving about 59% of the cable length compared with laying cable on every road. (3) The **Fractional Knapsack** greedy method decides how a relief truck of limited capacity should be loaded to carry the maximum priority value. (4) **Huffman coding** compresses emergency broadcast messages by about half. A live benchmark of **Bubble, Merge and Quick sort** verifies the Big-Oh analysis experimentally.

Every algorithm is visualised step by step on an interactive map, making the project easy to demonstrate and to explain. The project connects the theory of Unit 1 (analysis, sorting) and Unit 2 (greedy method, spanning trees, shortest paths, Huffman trees) of the syllabus with a practical, working system.

Keywords: Dijkstra, Kruskal, Prim, Greedy method, Fractional Knapsack, Huffman coding, Time complexity, Python.

TABLE OF CONTENTS

ACKNOWLEDGEMENT **3**
ABSTRACT **4**
1. INTRODUCTION **7**
1.1 Problem Statement 7
1.2 Why This Project? 7
1.3 Overview of the System 7
2. AIM **8**
3. OBJECTIVES **8**
4. TOOLS AND SYSTEM REQUIREMENTS **8**
4.1 Software Requirements 8
4.2 Hardware Requirements 8
4.3 How to Run 9
5. SYLLABUS MAPPING AND ALGORITHMS USED **9**
5.1 Dijkstra's Algorithm (Greedy) 9
5.2 Kruskal's Algorithm 10
5.3 Prim's Algorithm 11
5.4 Fractional Knapsack 11
5.5 Huffman Coding 12
5.6 Merge Sort, Quick Sort and Bubble Sort 12
6. SYSTEM DESIGN **13**
6.1 Architecture 13
6.2 Modules of the Application 13
6.3 Data Used 14
7. IMPLEMENTATION **14**
7.1 Dijkstra's Algorithm 15
7.2 Kruskal's Algorithm with Union–Find 15
7.3 Fractional Knapsack 15
7.4 Huffman Coding 16
7.5 Testing 16
8. SCREENSHOTS AND RESULTS **17**
8.1 Observations 19
9. COMPLEXITY SUMMARY **20**
10. CONCLUSION AND FUTURE SCOPE **20**
10.1 Conclusion 20
10.2 Future Scope 21
REFERENCES **21**

LIST OF FIGURES

Figure 1: Overall workflow of the CityNet Optimizer application 7
Figure 2: Flowchart of Dijkstra's algorithm with a min-heap 10
Figure 3: Edges examined by Kruskal's algorithm in increasing order of weight 11
Figure 4: Flowcharts of Kruskal's and Prim's algorithms 11
Figure 5: Huffman tree for "SEND HELP TO AMRITSAR" with the code of each leaf 12
Figure 6: Layered architecture of CityNet Optimizer 13
Figure 7: Road network of ten Punjab cities (edge labels are distances in km) 14
Figure 8: Home screen – Emergency Route module with road network 17
Figure 9: Dijkstra's algorithm – shortest route Ludhiana → Amritsar (142 km) 17
Figure 10: Dijkstra's algorithm – another route, Bathinda → Chandigarh 17
Figure 11: Kruskal's algorithm – minimum fibre network of 564 km 18
Figure 12: Prim's algorithm – the same minimum cost of 564 km 18
Figure 13: Fractional Knapsack – relief truck of 120 kg loaded for a maximum value of 890 18
Figure 14: Value/weight ratios and the fraction of each item loaded 19
Figure 15: Huffman tree and codes – 168 bits compressed to 79 bits 19
Figure 16: Sorting benchmark – Bubble sort grows quadratically 19
Figure 17: Theoretical growth rates (left) and measured sorting times (right) 20

LIST OF TABLES

Table 1: Software requirements 8
Table 2: Minimum hardware requirements 8
Table 3: Syllabus mapping of the algorithms used 9
Table 4: Order in which Dijkstra settles the cities (source: Ludhiana) 10
Table 5: Relief items ranked by value/weight ratio (truck capacity 120 kg) 12
Table 6: Layers and their responsibilities 13
Table 7: Roads used (approximate distance in km) 14
Table 8: Test cases and results 16
Table 9: Time and space complexity of the algorithms used 20

1. INTRODUCTION

An algorithm is a finite sequence of well-defined steps that solves a problem. The Design and Analysis of Algorithms (DAA) subject teaches how to design such algorithms using standard paradigms – divide and conquer, greedy method, dynamic programming and backtracking – and how to measure their efficiency using time and space complexity.

This mini project applies those ideas to a believable scenario: the administration of a regional "smart city" network. The administration needs to (a) send ambulances along the shortest road, (b) connect all cities with the least amount of optical-fibre cable, (c) load limited-capacity trucks with the most valuable relief material, and (d) compress emergency broadcast messages. Each need maps directly to a classical algorithm in our syllabus.

1.1 Problem Statement

Given a road network of cities with known distances, design and implement an application that finds shortest routes, builds a minimum-cost spanning network, optimally loads a vehicle with divisible relief goods and compresses messages, together with a sorting benchmark – while visualising every step of the algorithms.

1.2 Why This Project?

- **It is realistic** – every module solves a problem that real municipalities face.
- **It is visual** – the graph is drawn and the algorithms are animated, so a viewer understands the logic without reading code.
- **It is explainable** – each module uses one clean algorithm with a clear complexity.
- **It includes an experiment** – the benchmark module proves $O(n^2)$ vs $O(n \log n)$ with real timings.

1.3 Overview of the System

The application opens with a left-hand menu of five modules. Choosing a module loads its controls on the right, draws the road map or chart in the centre, and prints a step-by-step log. Figure 1 shows how a request travels from the user to the algorithm and back to the screen.

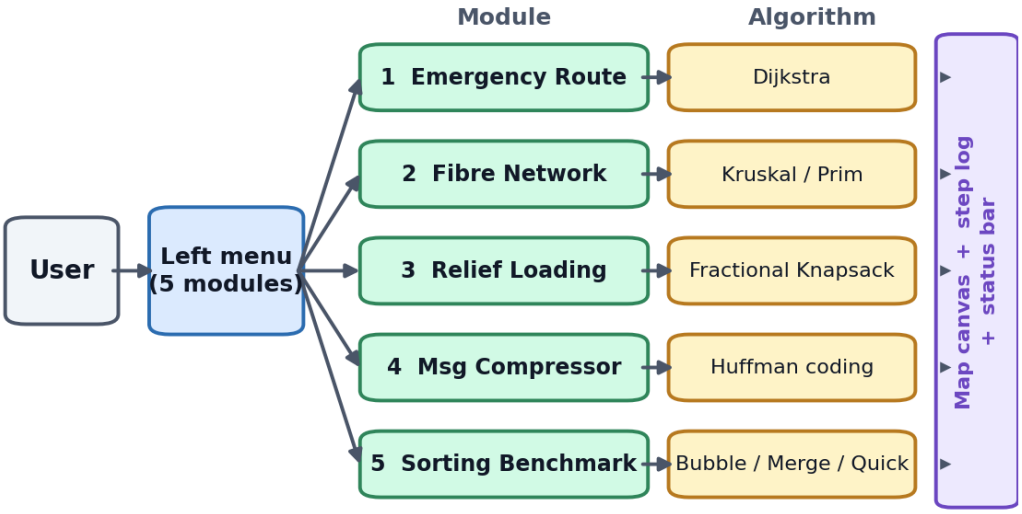


Figure 1: *Overall workflow of the CityNet Optimizer application*

2. AIM

To design and implement a Smart City Network & Resource Planner that applies classical algorithm design techniques – Greedy method and Sorting – to solve practical routing, network-design and resource-allocation problems, and to analyse their time complexity.

3. OBJECTIVES

- To model a city road network as a weighted, undirected graph using adjacency lists.
- To implement Dijkstra's algorithm and find the shortest emergency route between two cities.
- To implement Kruskal's and Prim's algorithms and compare the minimum spanning trees they produce.
- To implement the Fractional Knapsack greedy algorithm for optimal truck loading.
- To implement Huffman coding and build the Huffman tree for message compression.
- To implement Merge sort, Quick sort and Bubble sort and compare them.
- To verify the theoretical asymptotic complexity by measuring running times experimentally.
- To build a clear, attractive graphical interface that animates each algorithm.

4. TOOLS AND SYSTEM REQUIREMENTS

4.1 Software Requirements

Table 1: *Software requirements*

Item	Details
Programming language	Python 3.8 or higher (tested on Python 3.12)
Editor / IDE	Python IDLE (any editor such as VS Code also works)
GUI library	Tkinter + ttk (comes pre-installed with Python – no installation needed)
Other libraries	heapq, random, time (Python standard library only)
Operating system	Windows 10 / 11, Linux or macOS

4.2 Hardware Requirements

Table 2: *Minimum hardware requirements*

Item	Minimum
Processor	Intel Core i3 / AMD Ryzen 3 or equivalent
RAM	4 GB
Disk space	Less than 5 MB
Display	1280 × 720 resolution or higher

Note: DevC++ and Java were also available, but Python was chosen because Tkinter gives a rich graphical interface with very little code, and Python's readable syntax makes the algorithms easy to explain in a viva.

4.3 How to Run

1. Install Python from python.org (tick "Add Python to PATH").
2. Open `citynet.py` in IDLE and press F5 (or run `python citynet.py` in a terminal).
3. Use the five modules from the left menu.

5. SYLLABUS MAPPING AND ALGORITHMS USED

Table 3 shows which syllabus topic is used in each module of the project.

Table 3: *Syllabus mapping of the algorithms used*

Algorithm	Syllabus chapter	Used in the project for	Time complexity
Dijkstra's algorithm	Unit 2 – Ch. 2.4 Shortest paths	Emergency route between two cities	O((V + E) log V)
Kruskal's algorithm	Unit 2 – Ch. 2.3 Minimum cost spanning trees	Cheapest fibre network (edge-based, with union–find)	O(E log E)
Prim's algorithm	Unit 2 – Ch. 2.3 Minimum cost spanning trees	Cheapest fibre network (vertex-based, with min-heap)	O(E log V)
Fractional Knapsack	Unit 2 – Ch. 2.2 Greedy method	Loading the relief truck	O(n log n)
Huffman coding	Unit 2 – Ch. 2.5 Optimal tree problem	Compressing emergency messages	O(n log n)
Merge sort	Unit 1 – Ch. 1.4 Sorting (divide & conquer)	Sorting benchmark	O(n log n)
Quick sort / Bubble sort	Unit 1 – Ch. 1.4 Sorting	Sorting benchmark	O(n log n) avg / O(n^2)
Asymptotic notations	Unit 1 – Ch. 1.3	Benchmark graph verifies Big-Oh	–
Graph, Heap (priority queue)	Unit 1 – Ch. 1.5; Unit 2 – Ch. 2.6	Adjacency-list graph; min-heap in Dijkstra, Prim and Huffman	–

5.1 Dijkstra's Algorithm (Greedy)

Dijkstra's algorithm finds the shortest path from one source to all other vertices of a graph with non-negative edge weights. It repeatedly picks the unvisited vertex with the smallest known distance (greedy choice) and relaxes its neighbouring edges. A min-heap (priority queue) is used to pick that vertex quickly. In the project, the visited order is printed in the log panel and the final route is animated in green on the map. Figure 2 shows the flow of the algorithm.

Worked example. For an ambulance starting at Ludhiana with Amritsar as the hospital, the cities are settled in the order shown in Table 4 and the shortest route is Ludhiana → Jalandhar → Amritsar (62 + 80 = 142 km). The search stops as soon as the destination is settled.

Table 4: *Order in which Dijkstra settles the cities (source: Ludhiana)*

Step	City settled	Distance (km)	Step	City settled	Distance (km)
1	Ludhiana	0	6	Hoshiarpur	102
2	Jalandhar	62	7	Ferozepur	107
3	Moga	62	8	Patiala	110
4	Sangrur	80	9	Amritsar	142
5	Chandigarh	100	–	–	–

A second query, Bathinda → Chandigarh, returns the route Bathinda → Patiala → Chandigarh with a total of 90 + 65 = 155 km.

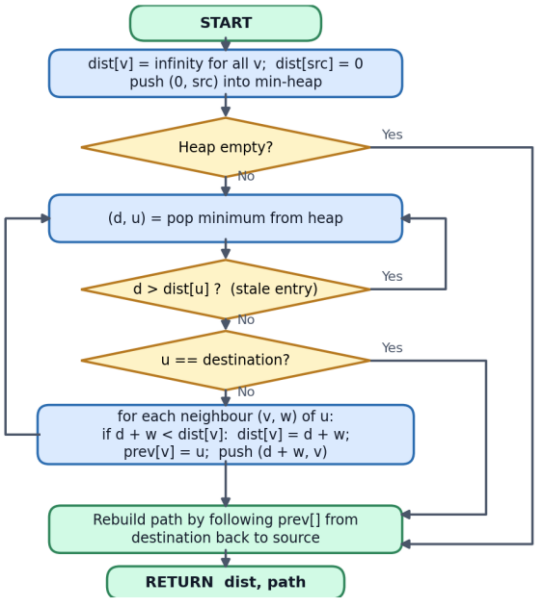


Figure 2: *Flowchart of Dijkstra's algorithm with a min-heap*

5.2 Kruskal's Algorithm

Kruskal's algorithm builds a minimum spanning tree by sorting all edges in increasing order of weight and adding an edge only if it does not form a cycle. Cycle detection is done with a Disjoint-Set (Union–Find) structure with path compression. Accepted edges are drawn in green and rejected edges in dashed red, so the cycle-avoidance rule is visible. Of the 17 roads, 9 are accepted and 8 are rejected (Figure 3).

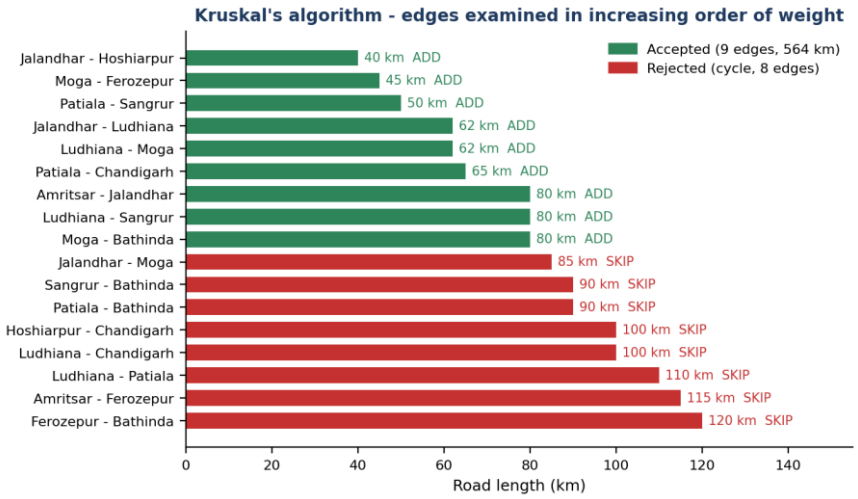


Figure 3: *Edges examined by Kruskal's algorithm in increasing order of weight*

5.3 Prim's Algorithm

Prim's algorithm grows the tree from a start vertex (Ludhiana), always adding the cheapest edge that connects the tree to a new vertex. It adds the roads Ludhiana–Jalandhar, Jalandhar–Hoshiarpur, Ludhiana–Moga, Moga–Ferozepur, Jalandhar–Amritsar, Ludhiana–Sangrur, Sangrur–Patiala, Patiala–Chandigarh and Moga–Bathinda, in that order. Both Kruskal and Prim give the same total cost (564 km), which is a useful check of correctness. Figure 4 compares the two procedures.

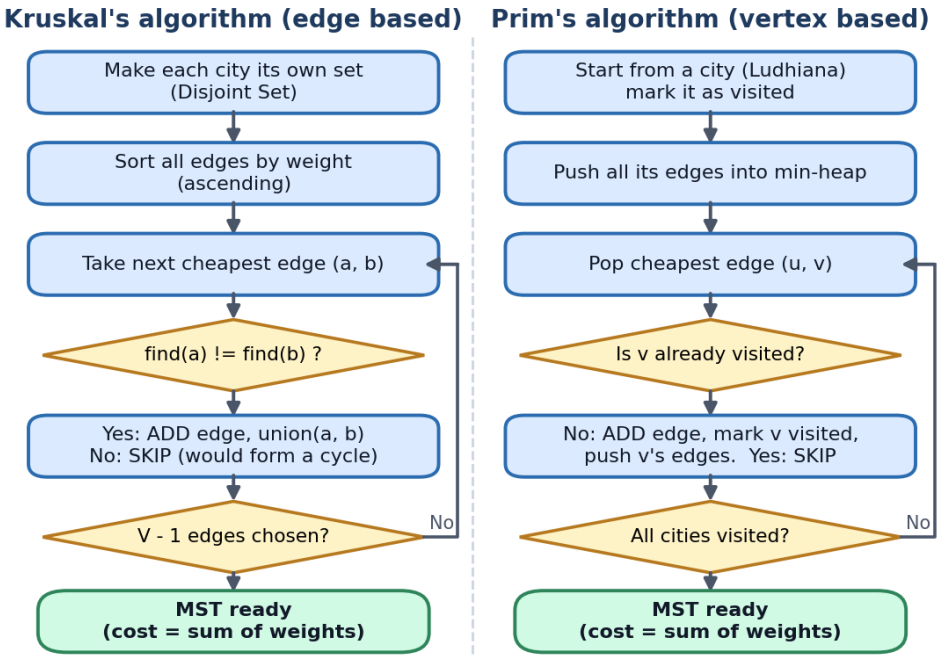


Figure 4: *Flowcharts of Kruskal's and Prim's algorithms*

5.4 Fractional Knapsack

Items are divisible, so the greedy strategy is optimal: compute the value-to-weight ratio of each item, sort in decreasing order, take whole items while they fit, and take a fraction of the next item to fill the remaining capacity. For a 120 kg truck the ranking and the quantity loaded are shown in Table 5.

Table 5: *Relief items ranked by value/weight ratio (truck capacity 120 kg)*

Item	Weight (kg)	Priority value	Ratio	Taken	Value
Medicines	10	300	30.00	100%	300.0
Food packets	40	280	7.00	100%	280.0
Blankets	30	150	5.00	100%	150.0
Tents	60	240	4.00	67%	160.0
Generators	80	320	4.00	0%	0.0
Water cans	50	100	2.00	0%	0.0
Total	**120 kg**				**890.0**

5.5 Huffman Coding

Huffman coding is a greedy algorithm for the optimal tree problem. Characters are placed in a min-heap by frequency; the two least frequent nodes are repeatedly removed and merged into a new node whose frequency is their sum, until a single tree remains. Moving left adds the bit 0 and moving right adds the bit 1, so frequent characters get short codes and rare characters get long ones. No code is a prefix of another, so the compressed message can be decoded without ambiguity. The project draws the complete Huffman tree and shows the number of bits saved. Figure 5 is the tree built for the message "SEND HELP TO AMRITSAR".

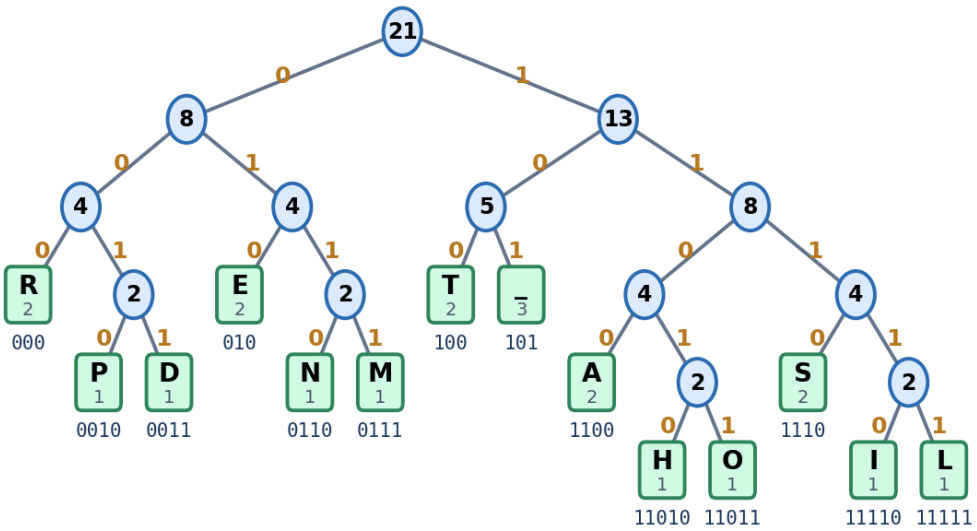


Figure 5: *Huffman tree for "SEND HELP TO AMRITSAR" with the code of each leaf*

5.6 Merge Sort, Quick Sort and Bubble Sort

Merge sort (divide and conquer) splits the list into halves, sorts them recursively and merges the results in O(n log n) time. Quick sort partitions around a pivot and has O(n log n) average time, while Bubble sort repeatedly swaps adjacent elements and needs $O(n^2)$ time. The benchmark module times all three on random lists of increasing size and plots the results, which verifies the asymptotic analysis of Unit 1 experimentally.

6. SYSTEM DESIGN

6.1 Architecture

The program is separated into two layers: an algorithm layer of pure functions (no GUI code) and a presentation layer (Tkinter) that calls them and draws the results. The data constants form a third, passive layer. This keeps the algorithms easy to test and explain.

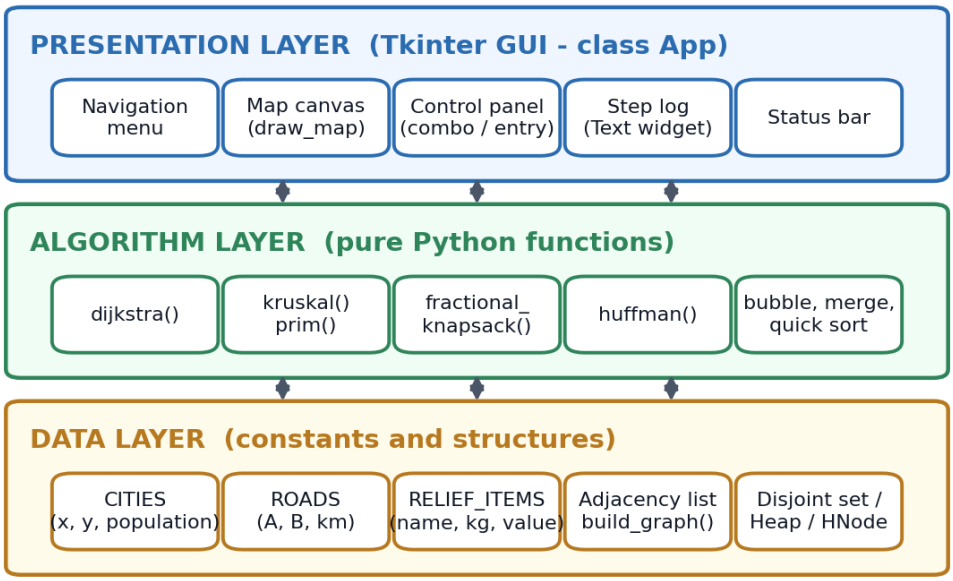


Figure 6: *Layered architecture of CityNet Optimizer*

Table 6: *Layers and their responsibilities*

Layer / Module	Responsibility
Data	CITIES (name, x, y, population), ROADS (city A, city B, km), RELIEF_ITEMS (name, weight, value)
Graph	build_graph() creates an adjacency list: {city: [(neighbour, distance), …]}
Algorithm layer	dijkstra(), kruskal(), prim(), fractional_knapsack(), huffman(), merge_sort(), quick_sort(), bubble_sort()
GUI layer	class App – navigation menu, map canvas, control panel, step-by-step log

6.2 Modules of the Application

- **Module 1 – Emergency Route:** choose a start and a destination; Dijkstra's route is animated.
- **Module 2 – Fibre Network:** choose Kruskal or Prim; the spanning tree is built edge by edge.
- **Module 3 – Relief Loading:** enter truck capacity; a load bar and table show what is carried.
- **Module 4 – Message Compressor:** type a message; the Huffman tree, codes and bits saved are shown.
- **Module 5 – Sorting Benchmark:** plots running time of three sorting algorithms against input size.

6.3 Data Used

Ten cities and seventeen roads are used (city populations are stored in the program but not used by the current modules). The distances are approximate road distances in kilometres and the populations are rounded figures in lakh; they are included for demonstration and are not official survey values. The data lists can be edited at the top of the program to model any other region. Figure 7 shows the resulting graph and Table 7 lists all seventeen roads.

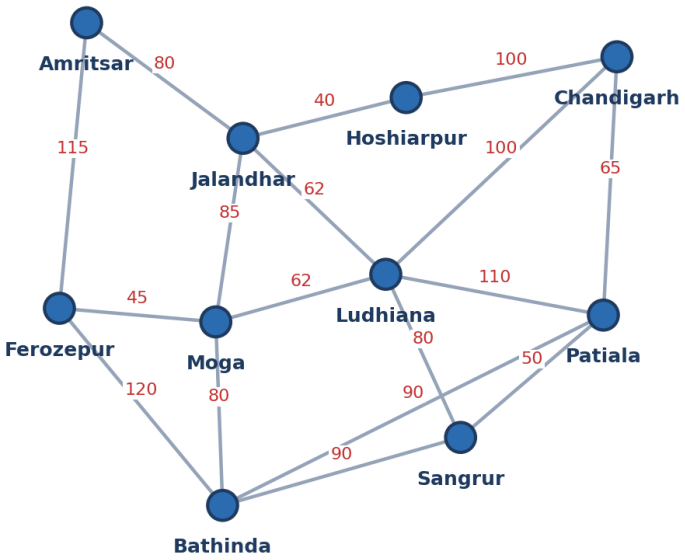


Figure 7: *Road network of ten Punjab cities (edge labels are distances in km)*

Table 7: *Roads used (approximate distance in km)*

Road	km	Road	km
Amritsar – Jalandhar	80	Ludhiana – Sangrur	80
Amritsar – Ferozepur	115	Moga – Ferozepur	45
Jalandhar – Hoshiarpur	40	Moga – Bathinda	80
Jalandhar – Ludhiana	62	Patiala – Chandigarh	65
Jalandhar – Moga	85	Patiala – Sangrur	50
Hoshiarpur – Chandigarh	100	Sangrur – Bathinda	90
Ludhiana – Chandigarh	100	Patiala – Bathinda	90
Ludhiana – Moga	62	Ferozepur – Bathinda	120
Ludhiana – Patiala	110		

7. IMPLEMENTATION

The complete source code is in the file `citynet.py` (659 lines). The key algorithm functions are reproduced below exactly as they appear in the program; Prim's algorithm, the sorting functions and the GUI code are in the same file.

7.1 Dijkstra's Algorithm

```
def dijkstra(graph, src, dst):
    """Greedy single-source shortest path with a min-heap.  O((V+E) log V)"""
    dist = {v: float("inf") for v in graph}
    prev, dist[src] = {}, 0
    pq, order = [(0, src)], []
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        order.append((u, d))
        if u == dst:
            break
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v], prev[v] = d + w, u
```

```
                heapq.heappush(pq, (d + w, v))
    path, node = [], dst
    while node != src and node in prev:
        path.append(node)
        node = prev[node]
    path.append(src)
    return dist[dst], path[::-1], order
```

7.2 Kruskal's Algorithm with Union–Find

```
class DisjointSet:
    def __init__(self, items):
        self.p = {i: i for i in items}

    def find(self, x):
        while self.p[x] != x:
            self.p[x] = self.p[self.p[x]]        # path compression
            x = self.p[x]
        return x

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        self.p[ra] = rb
        return True
```

```
def kruskal(nodes, edges):
    """Sort edges, add the cheapest that does not form a cycle.  O(E log E)"""
    ds, steps, total = DisjointSet(nodes), [], 0
    for a, b, w in sorted(edges, key=lambda e: e[2]):
        ok = ds.union(a, b)
        total += w if ok else 0
        steps.append((a, b, w, ok))
    return total, steps
```

7.3 Fractional Knapsack

```
def fractional_knapsack(items, capacity):
    """Greedy: take items in decreasing value/weight ratio.  O(n log n)"""
    ranked = sorted(items, key=lambda i: i[2] / i[1], reverse=True)
    left, value, plan = capacity, 0.0, []
    for name, wt, val in ranked:
        if left <= 0:
            plan.append((name, wt, val, 0.0))
            continue
        take = min(wt, left)
        frac = take / wt
        plan.append((name, wt, val, frac))
        value += val * frac
        left -= take
    return value, plan
```

7.4 Huffman Coding

```
def huffman(text):
    """Greedy: repeatedly merge the two least frequent nodes.  O(n log n)"""
    freq = {}
    for ch in text:
        freq[ch] = freq.get(ch, 0) + 1
    heap = [HNode(f, ch) for ch, f in freq.items()]
    heapq.heapify(heap)
    if len(heap) == 1:
        heap = [HNode(heap[0].freq, None, heap[0], None)]
    while len(heap) > 1:
        a, b = heapq.heappop(heap), heapq.heappop(heap)
        heapq.heappush(heap, HNode(a.freq + b.freq, None, a, b))
    root, codes = heap[0], {}

    def walk(n, code):
        if n.ch is not None:
            codes[n.ch] = code or "0"
            return
        if n.left:
            walk(n.left, code + "0")
        if n.right:
            walk(n.right, code + "1")
    walk(root, "")
```

```
    return root, codes, freq
```

7.5 Testing

Each module was tested with known inputs. The results obtained from the program matched the values calculated by hand, as shown in Table 8.

Table 8: *Test cases and results*

Test	Expected	Obtained
Shortest route Ludhiana → Amritsar	Ludhiana – Jalandhar – Amritsar, 142 km	142 km ✓
Shortest route Bathinda → Chandigarh	Bathinda – Patiala – Chandigarh, 155 km	155 km ✓
MST cost: Kruskal vs Prim	Both equal	564 km both ✓
Knapsack, capacity 120 kg	Max value 890	890.0 ✓
Huffman: "SEND HELP TO AMRITSAR"	Decoded text equals original	168 bits → 79 bits, decode OK ✓
Sorted output vs Python sorted()	Identical	Asserted on every benchmark run ✓

8. SCREENSHOTS AND RESULTS

The following screenshots were taken from the running application. The charts that follow some of them explain the result in more detail.

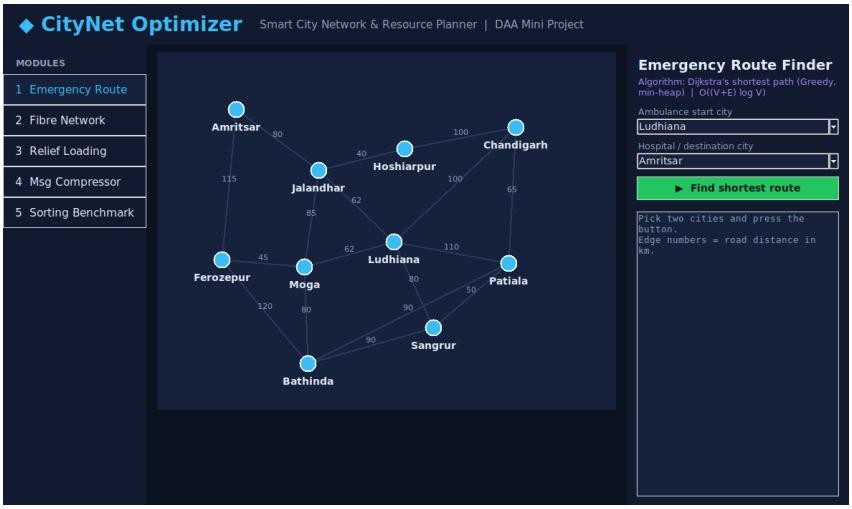


Figure 8: *Home screen – Emergency Route module with road network*

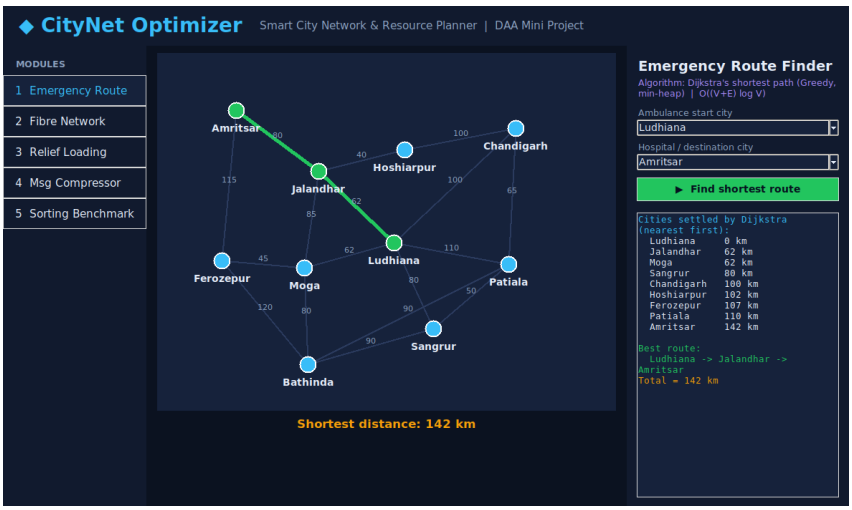


Figure 9: *Dijkstra's algorithm – shortest route Ludhiana → Amritsar (142 km)*

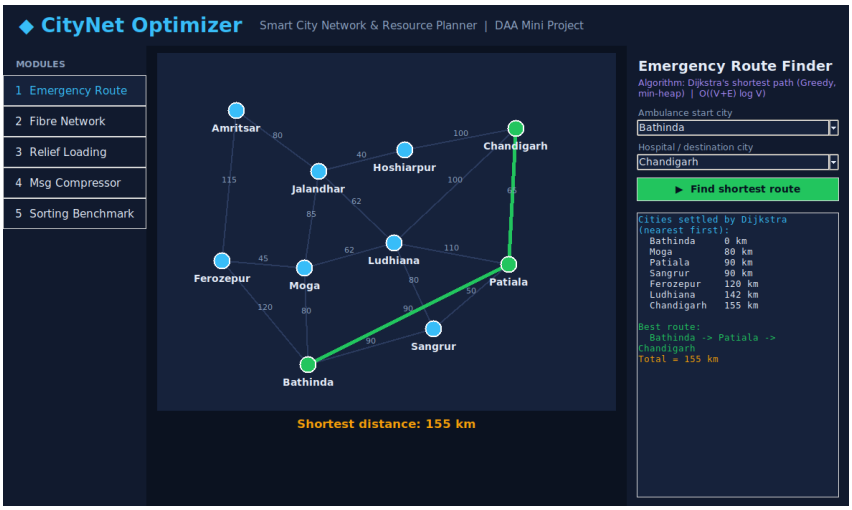


Figure 10: *Dijkstra's algorithm – another route, Bathinda → Chandigarh*

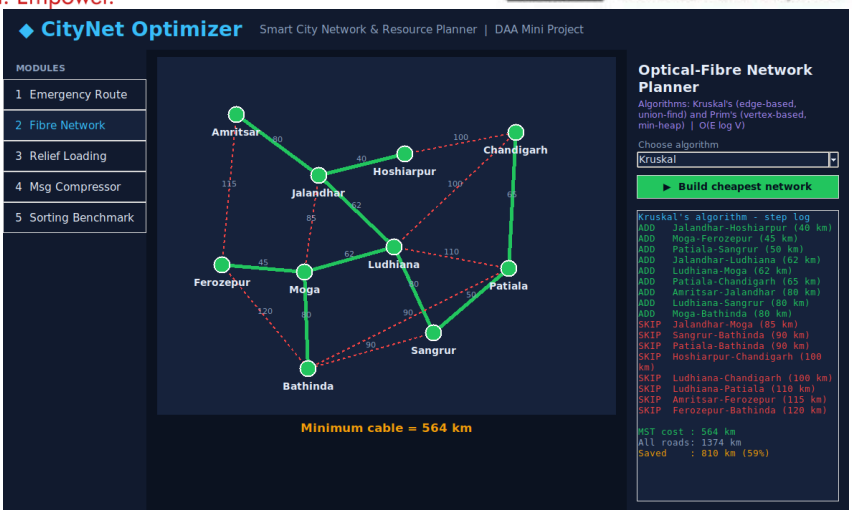


Figure 11: *Kruskal's algorithm – minimum fibre network of 564 km*

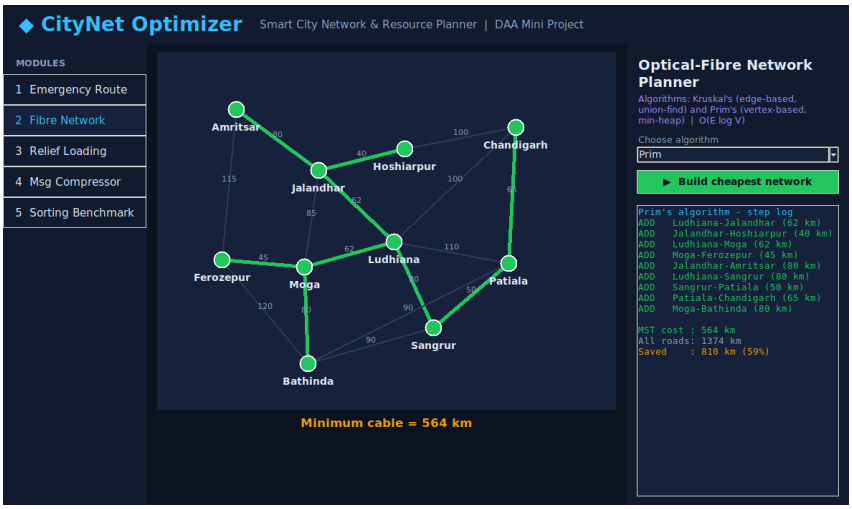


Figure 12: *Prim's algorithm – the same minimum cost of 564 km*

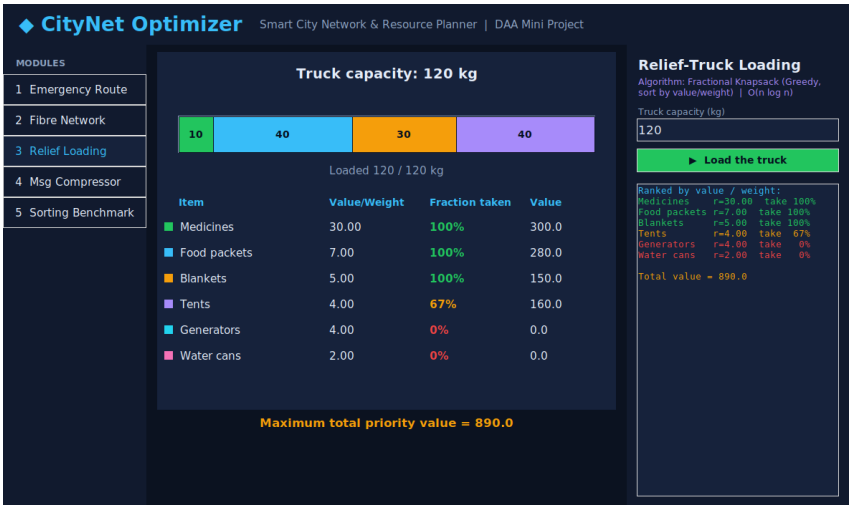


Figure 13: *Fractional Knapsack – relief truck of 120 kg loaded for a maximum value of 890*

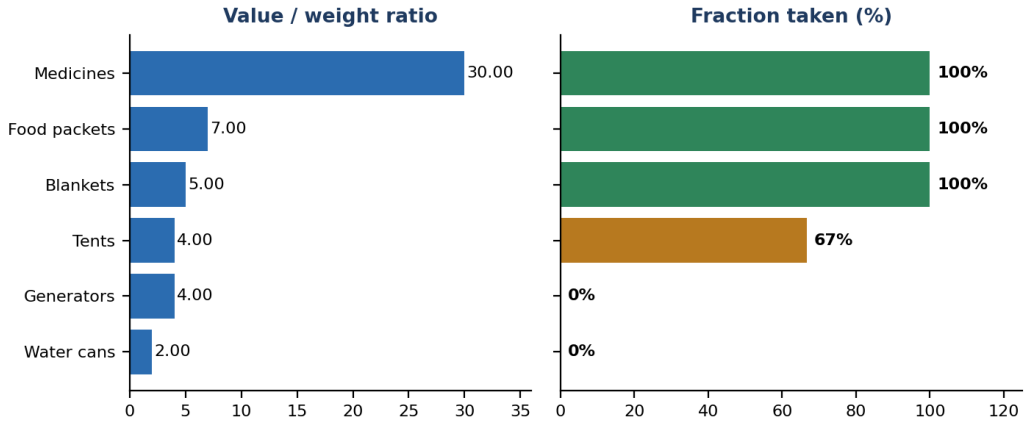


Figure 14: *Value/weight ratios and the fraction of each item loaded*

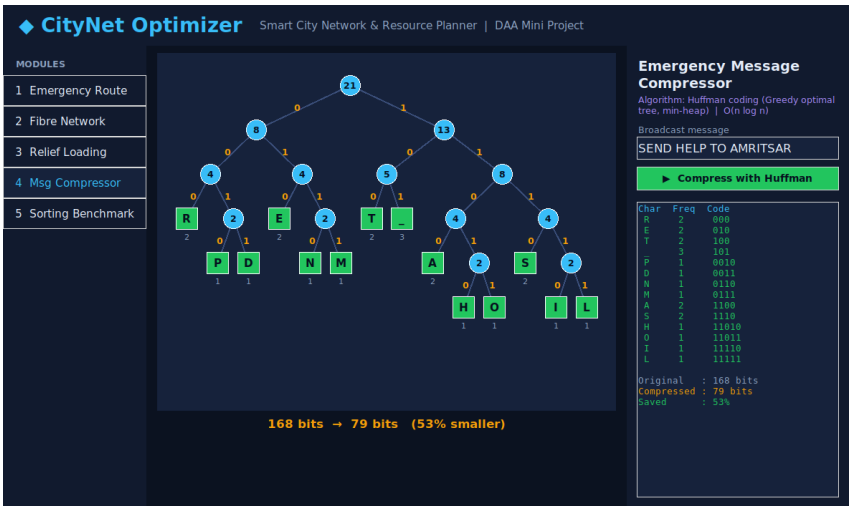


Figure 15: *Huffman tree and codes – 168 bits compressed to 79 bits*

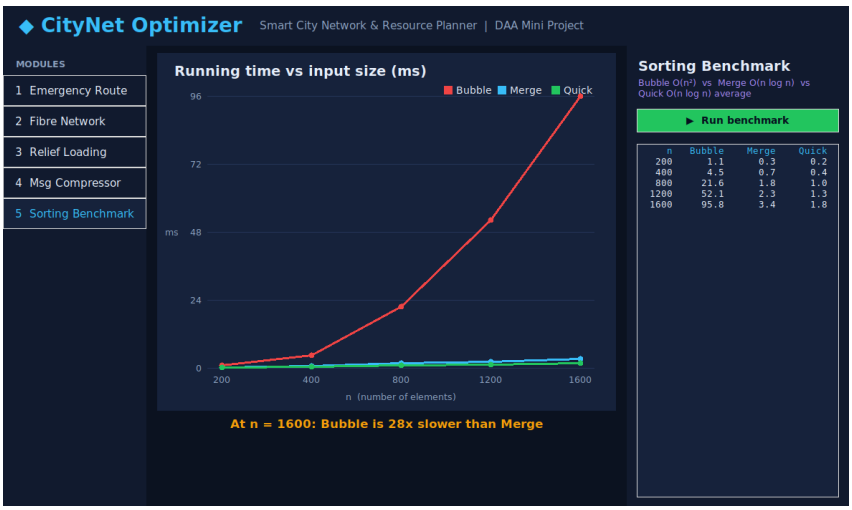


Figure 16: *Sorting benchmark – Bubble sort grows quadratically*

8.1 Observations

- Dijkstra settles cities in increasing distance order and stops as soon as the destination is reached.
- The spanning tree needs only 564 km of cable against 1374 km for all roads – a saving of 810 km (59%).

- Kruskal and Prim use different strategies but always give the same minimum cost, confirming their correctness.
- The knapsack loads medicines first (ratio 30), then food, blankets, and 67% of the tents.
- Huffman coding reduced the 21-character message from 168 bits to 79 bits (53% smaller); the most frequent letters received 3-bit codes and the rarest received 5-bit codes.
- In the benchmark, Bubble sort was about 28 times slower than Merge sort at n = 1600 (exact timings depend on the computer). This matches the $O(n^2)$ versus $O(n \log n)$ analysis.

9. COMPLEXITY SUMMARY

Table 9: *Time and space complexity of the algorithms used*

Algorithm	Time	Space	Strategy
Dijkstra (min-heap)	$O((V + E) \log V)$	$O(V)$	Greedy
Kruskal	$O(E \log E)$	$O(V)$	Greedy
Prim (min-heap)	$O(E \log V)$	$O(V)$	Greedy
Fractional Knapsack	$O(n \log n)$	$O(n)$	Greedy
Merge sort	$O(n \log n)$	$O(n)$	Divide & conquer
Quick sort	$O(n \log n)$ avg, $O(n^2)$ worst	$O(\log n)$	Divide & conquer
Bubble sort	$O(n^2)$	$O(1)$	Brute force
Huffman coding	$O(n \log n)$	$O(n)$	Greedy

The left chart of Figure 17 shows how the three growth rates diverge as the input grows. The right chart plots the running times measured by the benchmark module on the author's computer; Bubble sort follows the quadratic curve while Merge and Quick sort stay close to the n log n curve.

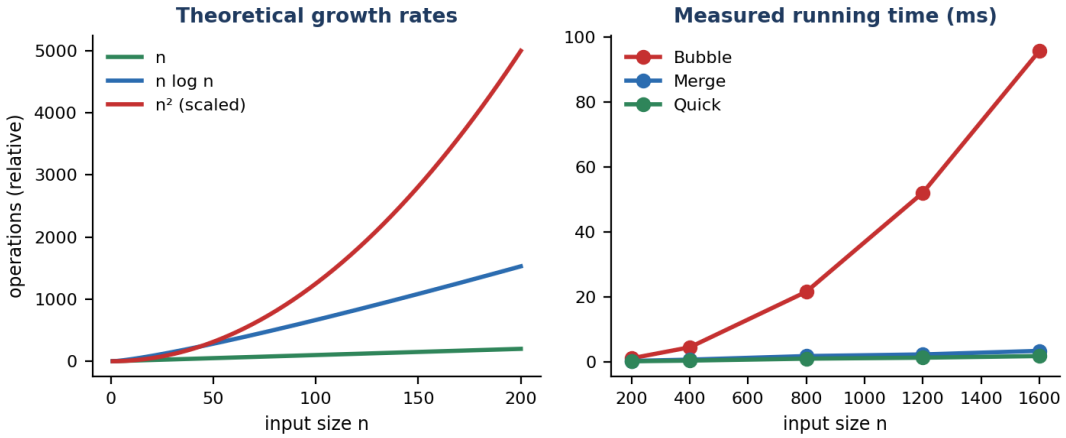


Figure 17: *Theoretical growth rates (left) and measured sorting times (right)*

10. CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

The CityNet Optimizer demonstrates that classical algorithms from the DAA syllabus can be combined into a practical and visual application. Dijkstra's algorithm gives dependable shortest routes, Kruskal's and Prim's algorithms reduce the network cost by almost 60%, the Fractional Knapsack method maximises the value carried by a limited-capacity truck, and Huffman coding shrinks emergency messages by about half. The benchmark confirmed experimentally that the choice of algorithm matters far more than the speed of the computer as the input grows.

Through this project I understood the greedy method, the importance of data structures such as heaps and disjoint sets, and how asymptotic notation predicts real performance.

10.2 Future Scope

- Add Floyd–Warshall (all-pairs shortest paths) and the Travelling Salesperson Problem (dynamic programming / branch and bound) for delivery tours.
- Add backtracking problems such as N-Queens (police checkpoint placement) and graph colouring (tower frequency planning).
- Allow users to add cities and roads by clicking on the map and load real data from a file.
- Use live map data (GPS / online map services) for real distances.

REFERENCES

[1] Anany Levitin, Introduction to the Design and Analysis of Algorithms, 3rd Edition, Pearson Education.

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, Introduction to Algorithms, 4th Edition, MIT Press.

[3] Ellis Horowitz, Sartaj Sahni and Sanguthevar Rajasekaran, Fundamentals of Computer Algorithms, 2nd Edition, Universities Press.

[4] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs", Numerische Mathematik, Vol. 1, 1959.

[5] Python Software Foundation, Python 3 documentation – tkinter (Python interface to Tcl/Tk): https://docs.python.org/3/library/tkinter.html

[6] Python Software Foundation, Python 3 documentation – heapq (heap queue algorithm): https://docs.python.org/3/library/heapq.html

[7] Class notes and syllabus of Design and Analysis of Algorithms, Chandigarh University.